Laboratorio 2: Primera Aplicación Nativa y Control de Versiones con GitHub

Este laboratorio está diseñado para estudiantes que ya completaron la estandarización técnica (Laboratorio 1) y ahora tendrán su primer contacto con la creación de proyectos y la estructura de archivos en Android Studio.

1. OBJETIVOS DE APRENDIZAJE

- Implementar la técnica de **Scaffolding** (uso de plantillas) para nuevos proyectos.
- Identificar la anatomía de un proyecto Android profesional.
- Gestionar el ciclo de vida del código mediante **Git y GitHub**.
- Documentar cambios en el código a través de múltiples *commits*.

2. REQUISITOS PREVIOS

- Haber completado el **Laboratorio 1** (Estandarización).
- Contar con conexión a internet para la vinculación con la nube.

3. PASO A PASO

Paso 1: Creación del Proyecto desde la "Plantilla Maestra"

Para garantizar que mantenemos el "blindaje" (Java 21, API 36 y Version Catalog) configurado en el Lab 1, no crearemos un proyecto desde cero, sino que clonaremos nuestra base:

1. **Copia Física:** En su explorador de archivos, copie la carpeta de su Laboratorio 1 y péguela con el nombre RDA1_Laboratorio2_NombreApellido.
2. **Limpieza:** Borre las carpetas ocultas .gradle y build dentro de la nueva carpeta.
3. **Renombrado en el IDE**: Abra la carpeta en Android Studio. Vaya al archivo settings.gradle.kts y cambie el nombre del proyecto: rootProject.name = "MiPrimerAppAndroid"
4. **Sincronización:** Presione el icono del elefante (Sync Project) para validar que el entorno sigue siendo funcional.

Paso 2: Preparación del Entorno GitHub

Antes de subir código, debemos preparar la infraestructura de identidad en la nube:

1. **Registro:** Ingrese a GitHub.com y cree una cuenta personal. Use su correo institucional para obtener beneficios de estudiante.
2. **Instalación de Git:** Descargue e instale Git for Windows desde git-scm.com.
3. **Configuración de Identidad en la PC:** Abra una terminal (CMD o PowerShell) y ejecute los siguientes comandos para que sus aportes tengan nombre:
    - git config --global user.name "Su Nombre y Apellido"
    - git config --global user.email "su_correo@ejemplo.com"

Paso 3: Vinculación con GitHub

Guardaremos el estado actual del proyecto antes de hacer cambios:

1. **Crear Repositorio Local:** En Android Studio, vaya a **VCS > Create Git Repository**. Seleccione la carpeta raíz de su proyecto. Los archivos se pondrán en color **marrón/rojo** (indicando que Git los detectó, pero aún no los "conoce"). Si Android Studio le indica que Git no está instalado, cierre Android y vuelva a abrirlo.
2. **Primer Commit:** Abra la pestaña **Commit** (icono de check  a la izquierda).
    - Marque todos los archivos y escriba el mensaje: "Initial Commit: Proyecto base estandarizado".
    - Presione **Commit**. Los archivos se pondrán blancos/negros.
3. **Vincular a la Nube:**
    - Cree un nuevo repositorio vacío en su cuenta de GitHub web (sin README).
    - Copie la URL del repositorio (termina en `.git`).
    - En Android Studio, vaya a **Git > Manage Remotes**. Haga clic en el símbolo + y pegue la URL.

Paso 4: Modificación del Código y Personalización

1. Abra el archivo **MainActivity.kt**.
2. Localice la función **@Composable Greeting**.
3. Modifique la línea del componente **Text**:

```
Text(
    text = "Bienvenido al curso 2026, $name",
    color = Color.Blue,
    style = MaterialTheme.typography.headlineMedium
)
```

4. Ejecute la **app** en el emulador para verificar el cambio.

Paso 5: Tarea de Control de Versiones (Segundo Commit)

Para que en GitHub se vean las dos versiones, debemos registrar este nuevo cambio:

1. Vuelva a la pestaña **Commit**. Notará que solo aparece el archivo **MainActivity.kt** en color **azul** (modificado).
2. Seleccione el archivo **MainActivity.kt.** Puede que aparezcan otros archivos si usted los modificó también, selecciónelos si es este su caso.
3. Escriba el mensaje de **commit**: "Update: Se personaliza el mensaje de bienvenida y color" y presione **Commit**.
4. Presione el botón **Push** (flecha verde hacia arriba en la esquina superior izquierda al lado del nombre de la Aplicación o Ctrl+Shift+K) para subir este segundo cambio a GitHub. Puede que le salga un mensaje de advertencia indicando que hay algunas alertas en la compilación del código, seleccione **Push Anyway**. Si este mensaje de alerta indica que hay errores en el código corríjalos antes de realizar el **Push**.

Nota: si aún no está Logueado en GitHub, el momento de hacer la primera operación de **Push**, se le mostrará un mensaje de **Log In to GitHub**. Selecciones **Log In via GitHub** y siga el proceso de Log In desde la página de Git Hub.

Paso 6: Recuperación de Versiones (Rollback / Checkout)

El verdadero poder de Git es actuar como una "máquina del tiempo". En este paso, aprenderá a recuperar la primera versión del programa si algo sale mal.

1. **Abrir el historial local:** En la parte inferior de Android Studio, haga clic en la pestaña **Git** (junto a Terminal). Verá una lista de tus commits (*Initial Commit* y *Update*).
2. **Visualizar el pasado:** Haga clic derecho sobre su primer commit (**Initial Commit**) y seleccionar la opción **"Checkout Revision..."**.
    - **Observación:** Verá que el código en su editor vuelve a ser el original (texto en negro, sin su personalización). Git ha reescrito sus archivos locales para que coincidan con esa versión.
3. Vuelva a sincronizar el proyecto haciendo clic en el **Elefante** y ejecute nuevamente la **MainActivity** (Shift + F10)
    - **Observación:** Verá en el emulador cómo se vuelve a abrir la aplicación, pero esta vez el mensaje mostrado al usuario es simplemente “**Hello Android**”
4. **Regresar al presente:** Para volver a su trabajo más reciente, en la misma lista de commits, buscar el último (**Update**), haga clic derecho y seleccionar de nuevo

"Checkout Revision" o seleccionar su rama principal (master o main) en la esquina inferior derecha y dar a **Checkout**.

5. Vuelva a sincronizar el proyecto haciendo clic en el **Elefante** y ejecute nuevamente la **MainActivity** (Shift + F10)
    - **Observación:** Verá en el emulador cómo se vuelve a abrir la aplicación, pero esta vez el mensaje mostrado al usuario es "**Bienvenido al curso 2026, Android**"
6. **Uso de "Revert":** Si por error borró una función y quiere deshacer solo ese cambio sin moverse de versión, puede hacer clic derecho en el archivo y seleccionar **Git > local History > Show History**.

Nota: Esta función es vital cuando una actualización rompe la aplicación. Nunca borre y vuelva a escribir; se debe usar Git para ir al momento en que el código funcionaba.

---

4. TAREA EN GITHUB: VERIFICACIÓN DE VERSIONES

El estudiante debe demostrar que sabe navegar por el historial de su código:

1. Ingrese a su repositorio en **GitHub.com**.
2. Haga clic en el botón que dice **"2 commits"** (ubicado en la parte superior derecha de la lista de archivos).
3. Debe visualizar una lista con los dos mensajes enviados:
    - *Commit 1:* Initial Commit...
    - *Commit 2:* Update: Se personaliza el mensaje...
4. Haga clic en el código del segundo commit para ver el **"Diff"** (la comparación donde GitHub marca en rojo lo que borró y en verde lo que agregó).

---

5. ENTREGABLES

Deberá subir un archivo PDF que contenga:

1. **Informe:** con capturas de pantallas pertinentes

2. **Captura de Pantalla 1:** El emulador corriendo la aplicación con el cambio de texto y color.
3. **Captura de Pantalla 2:** Su historial de **Commits en GitHub**, donde se vean claramente los dos registros realizados.
4. **Captura de Pantalla 3:** Una imagen de la pestaña **Git Log** de Android Studio donde se vea el historial de sus commits y la marca de la versión que tienes activa en ese momento.
5. **Enlace (URL):** El link público a su repositorio de GitHub para revisión del docente.

---

6. RÚBRICA DE EVALUACIÓN

- No olvide revisar la rúbrica de evaluación, en el EVA, para complementar su informe.